

SMART INDIA HACKATHON 2026

User Manual

AI-Assisted Virtual Camera Tracking System for Coarse Alignment of Mobile FSOC Terminals

Problem Statement Code	SIH26169
Problem Statement	Development of an AI-Based Virtual Camera Tracking System for Coarse Alignment of Mobile Free Space Optical Communication (FSOC) Terminals
Organization	Indian Space Research Organisation (ISRO)
Team Name	404 Not Found
Team ID	155387
Institute	Tecnia Institute of Advanced Studies
Application	Python desktop simulation and tracking dashboard

Submission Document

Document Purpose

This manual provides the complete operating procedure for the SIH26169 software prototype. It covers installation, launch, dashboard controls, tracking behavior, disturbance configuration, benchmark export, video input, troubleshooting, and recommended demonstration procedure.

Important Implementation Note

The submitted source is a Python desktop application. It does not require a web server, database, cloud account, external API, or GPU. The primary entry point is `dashboard.py`. The repository also contains standalone/reference scripts for acquisition and tracking. The current prototype uses computer vision, Kalman estimation, and PID control. It does not require a trained neural-network model.

Contents

1	System Requirements	1
1.1	Software	1
1.2	Hardware	1
2	Installation	1
2.1	Obtain the Source	1
2.2	Install Dependencies	1
2.3	Verify Installation	2
3	Launching the Application	2
4	Dashboard Layout	2
4.1	Camera Viewport	2
4.2	World Overview	2
4.3	Telemetry and Controls	2
5	Live Telemetry	3
6	Target Motion Control	3
7	Optical Disturbance Controls	4
7.1	Noise Mode	4
7.2	Atmosphere Mode	4
7.3	Camera Jitter	4
7.4	Platform Drift	4

7.5 Pan Speed	4
8 Presets	4
9 How Tracking Works	5
10 Operating States	5
10.1 SEARCHING	5
10.2 ACQUIRING	5
10.3 TRACKING	5
10.4 Target Loss	6
11 Benchmark Export	6
12 Video Input	6
13 Recommended Demonstration Procedure	7
14 Standalone Edge-Case Verification	7
15 Reset and Recovery	8
16 Troubleshooting	8
17 Project File Guide	8
18 Operational Limits	9
19 Data and Privacy	9
20 Final Operator Checklist	9
21 Conclusion	10

1 System Requirements

1.1 Software

The application requires:

- Python;
- OpenCV;
- NumPy; and
- Pillow.

The supplied dependency file is:

```
opencv-python
numpy
Pillow
```

A Windows desktop environment is recommended for the GUI demonstration because the primary application uses Tkinter.

1.2 Hardware

No specialized optical hardware is required. The system is designed as a software simulation. A normal modern desktop or laptop capable of running Python, OpenCV, and a graphical desktop is sufficient.

2 Installation

2.1 Obtain the Source

Extract the submitted project archive into a local directory. Open a terminal in the extracted project directory.

The main files should include:

```
dashboard.py
step2_camera.py
step4_search.py
step5_tracker.py
step4_dashboard.py
step4_edge_test.py
logger.py
requirements.txt
README.md
```

2.2 Install Dependencies

Run:

```
python -m pip install -r requirements.txt
```

If the machine has multiple Python installations, use the Python executable that will be used to launch the application:

```
py -m pip install -r requirements.txt
```

2.3 Verify Installation

Run the supplied edge-case test:

```
python step4_edge_test.py
```

A successful run reports target acquisition in frames and a PASS/FAIL comparison against the 2-second criterion. The included verification was observed to acquire the configured edge target in 9 frames, approximately 0.30 seconds at 30 FPS.

3 Launching the Application

Start the primary dashboard with:

```
python dashboard.py
```

The application opens a desktop dashboard. The simulation starts automatically.

The primary dashboard is the recommended application for demonstration because it integrates the camera feed, world view, controls, live telemetry, disturbance presets, and benchmark export in one window.

4 Dashboard Layout

The dashboard is organized into three functional areas.

4.1 Camera Viewport

The camera viewport displays the simulated 640×480 monochrome camera feed. The optical centre is the desired pointing location. When the beacon is visible, its measured/predicted location and tracking relationship to the centre are displayed.

4.2 World Overview

The world overview represents the 2000×2000 simulated environment at a reduced display scale. It shows the larger spatial context that cannot be seen in the camera viewport.

The live FOV footprint illustrates which part of the world is currently visible to the virtual camera.

4.3 Telemetry and Controls

The lower dashboard area contains:

1. live telemetry;

2. optical disturbance and PTZ controls; and
3. presets and benchmark actions.

5 Live Telemetry

The telemetry panel reports the following values.

Metric	Meaning
Operating State	Current acquisition/tracking state.
Pan / Tilt Angle	Virtual camera orientation relative to the world centre.
Tracking Error	Distance between estimated beacon position and optical centre in pixels.
Centroid RMSE	Root-mean-square error over recent tracked samples.
Lock Retention Rate	Percentage of tracked samples within the 10-pixel in-spec threshold.
Acquisition Time	Time from search start until the target meets the acquisition condition.
Loop Processing / FPS	Processing rate and approximate per-frame latency.

The implemented tracking-error criterion is:

$$e \leq 10 \text{ pixels.}$$

The displayed lock-retention target is:

$$L \geq 95\%.$$

The nominal acquisition target is:

$$t_{\text{acq}} \leq 2.0 \text{ seconds.}$$

6 Target Motion Control

The Target Motion selector provides five modes.

Horizontal Target moves left/right and reverses at the scene boundary.

Straight Line Target follows a diagonal bounded path.

Circular Target follows a circular orbit.

Figure 8 Target follows a lemniscate-like path.

Random Target performs a bounded random walk.

For a strong demonstration, use more than one motion mode. Horizontal motion is useful for a simple baseline; circular, figure-eight, and random motion are better stress cases.

7 Optical Disturbance Controls

7.1 Noise Mode

Available modes:

- None;
- Salt and Pepper;
- Gaussian; and
- Poisson.

Use the noise-level slider to increase the disturbance.

7.2 Atmosphere Mode

Available modes:

- Clear;
- Haze;
- Fog;
- Rain; and
- Low Light.

The atmosphere-severity slider controls the strength of the selected transformation.

7.3 Camera Jitter

Camera jitter injects random $\pm$ pixel displacement into the effective camera position for each frame. The supported control range is 0–20 pixels/frame.

7.4 Platform Drift

Platform drift introduces a slower, time-varying camera displacement. It is useful for representing carrier/platform movement.

7.5 Pan Speed

The pan-speed control is provided in degrees per second, with the dashboard range of approximately 5–10 degrees/s.

8 Presets

The main dashboard contains four quick presets.

Preset	Purpose
Clear Sky	Baseline with no atmospheric or sensor disturbance.
10% S&P Noise	Applies salt-and-pepper noise at the preset level.
Heavy Fog (75%)	Tests operation under strong simulated fog.
High Jitter (± 20 px)	Tests robustness against high-frequency camera displacement.

A recommended demonstration sequence is Clear Sky first, followed by Heavy Fog and High Jitter.

9 How Tracking Works

The application follows this sequence:

1. The virtual scene updates the moving beacon.
2. The camera extracts its current viewport.
3. Disturbances and noise are applied to the camera frame.
4. The detector searches for the beacon.
5. The Kalman filter predicts and/or updates the target state.
6. The tracking error relative to the image centre is calculated.
7. The PID controller converts that error into pan/tilt motion.
8. The camera position is updated.
9. Telemetry is written to the performance logger.
10. The next frame repeats the closed loop.

When the target is not visible, the spiral search scanner moves the camera through a coverage pattern. When the target is found, the system transitions into acquisition and then tracking.

10 Operating States

10.1 SEARCHING

The target is not currently locked. The camera executes the spiral search pattern.

10.2 ACQUIRING

A target has been detected, but it has not yet reached the required image-centre condition.

10.3 TRACKING

The estimated beacon position is within the implemented acquisition/lock condition. PID control continuously minimizes image-plane error.

10.4 Target Loss

During a short visual dropout, the Kalman prediction can be used to continue control. If the target remains unavailable beyond the configured 15-frame loss limit, the controller resets and the camera returns to search.

At 30 FPS, 15 frames represent approximately 0.5 seconds.

11 Benchmark Export

Use the benchmark-report export button in the dashboard after or during a demonstration session.

The logger creates two files in the `logs/` directory:

1. a timestamped telemetry CSV;
2. a timestamped summary JSON.

The CSV contains frame-level data such as state, motion, target position, camera position, pan/tilt, tracking error, Kalman error, FPS, latency, noise mode, and atmosphere mode.

The JSON contains session-level performance information including:

- duration;
- frames processed;
- acquisition time;
- reacquisition statistics when available;
- RMSE;
- average error;
- maximum error;
- lock retention;
- out-of-spec/target-loss percentage;
- average FPS; and
- average processing latency.

12 Video Input

The main dashboard provides an option to load an MP4 or compatible video file.

Procedure:

1. Open the application.
2. Select the video-load action.
3. Choose a supported video file.
4. The dashboard switches from the simulated camera source to the video source.

5. The application processes video frames in the tracking pipeline.

The simulator-specific world overview is not meaningful for an external video source because the external video does not expose the simulated world coordinates.

13 Recommended Demonstration Procedure

For an SIH presentation, use the following sequence.

Stage 1: Baseline

1. Launch `dashboard.py`.
2. Keep Clear Sky selected.
3. Use Horizontal target motion.
4. Show the world overview and camera viewport.
5. Explain that the camera begins away from the moving beacon.
6. Allow spiral acquisition to occur.
7. Point out the state transition into TRACKING.
8. Point out tracking error, lock retention, acquisition time, and FPS.

Stage 2: Motion Diversity

Change the target motion to Circular, Figure 8, or Random. Explain that the estimator and controller operate continuously rather than following a fixed scripted camera path.

Stage 3: Disturbance Stress Test

Apply Heavy Fog or High Jitter. Explain that the disturbance engine changes the camera observation while the tracker attempts to preserve the lock.

Stage 4: Performance Evidence

Export the benchmark report and show the generated CSV/JSON files. Use the actual values from the exported session rather than quoting invented results.

14 Standalone Edge-Case Verification

The repository includes:

```
step4_edge_test.py
```

Run:

```
python step4_edge_test.py
```

The test initializes the target at (640, 708) and runs the spiral acquisition for up to 300 frames. The included execution acquired the target in 9 frames:

$$9/30 = 0.30 \text{ s.}$$

The test therefore reports:

Spec <=2s: PASS

15 Reset and Recovery

If the simulation behaves unexpectedly:

1. press the dashboard Reset Simulation action;
2. return disturbances to Clear Sky;
3. select Horizontal motion;
4. verify that the telemetry returns to the initial search state;
5. allow the acquisition loop to restart.

Resetting clears the virtual camera source, controller state, Kalman state, search state, recent error history, and acquisition timer.

16 Troubleshooting

Problem	Action
ModuleNotFoundError	Run <code>python -m pip install -r requirements.txt</code> .
GUI does not open	Run from a graphical desktop session and verify Python's Tkinter support.
Application appears frozen	Check whether the window is paused or the host is under heavy load; reset the simulation.
Target is not acquired	Return to Clear Sky, reduce disturbances, and confirm the selected motion mode.
Tracking is unstable	Reduce disturbance severity or pan speed, then reset.
No report appears	Use the benchmark export action and verify write permissions for the project <code>logs/</code> directory.
Video does not load	Use an MP4 or another format supported by the local OpenCV build.
World overview is absent during video playback	This is expected because an external video does not provide simulated world coordinates.

17 Project File Guide

File	Role
<code>dashboard.py</code>	Primary integrated desktop application.
<code>step2_camera.py</code>	Scene, camera, motion, FOV, noise and atmosphere simulation.
<code>step4_search.py</code>	Spiral search and target visibility.
<code>step5_tracker.py</code>	Reference tracking implementation with OpenCV control panel.
<code>step4_dashboard.py</code>	Earlier acquisition-stage dashboard.
<code>step4_edge_test.py</code>	Headless acquisition verification.
<code>logger.py</code>	CSV and JSON performance logging.
<code>requirements.txt</code>	Runtime dependencies.
<code>README.md</code>	Repository-level overview.

18 Operational Limits

The user should interpret the application as a software demonstrator and algorithm-development environment.

The present implementation does not simulate:

- physical optical propagation;
- real lens aberration;
- real photodiode/CCD/CMOS sensor physics;
- a physical pan-tilt mechanism;
- real atmospheric turbulence propagation;
- communication modulation/demodulation; or
- a complete fine-pointing loop.

The 10-pixel tracking threshold is therefore a software benchmark in the simulated image plane, not a direct claim about physical angular pointing accuracy.

19 Data and Privacy

The application operates locally on the host computer. The supplied implementation does not transmit telemetry to a remote server or require an online account. Performance files are written locally to the project directory.

If an external video is loaded, the video is processed locally by OpenCV according to the source code.

20 Final Operator Checklist

Before an SIH demonstration:

1. Confirm the project directory is extracted correctly.
2. Confirm dependencies are installed.
3. Run `python step4_edge_test.py` once.
4. Launch `python dashboard.py`.
5. Verify the camera viewport is updating.
6. Verify the world overview is visible.
7. Run Clear Sky baseline.
8. Demonstrate at least one non-horizontal motion.
9. Demonstrate at least one disturbance.
10. Show live error and lock telemetry.
11. Export a benchmark report.
12. Keep the generated CSV and JSON with the submission evidence.

21 Conclusion

The application provides a complete software workflow for demonstrating coarse virtual camera alignment for FSOC: scene generation, target motion, search, detection, estimation, feedback control, disturbance injection, visualization, and performance logging.

The primary operator entry point is `dashboard.py`. The included edge-case test has been executed successfully, and the benchmark subsystem is available for generating session-specific evidence.